



Industrial Machine Learning on Hadoop and Spark

Maks Nakhodnov, Bremen 2025

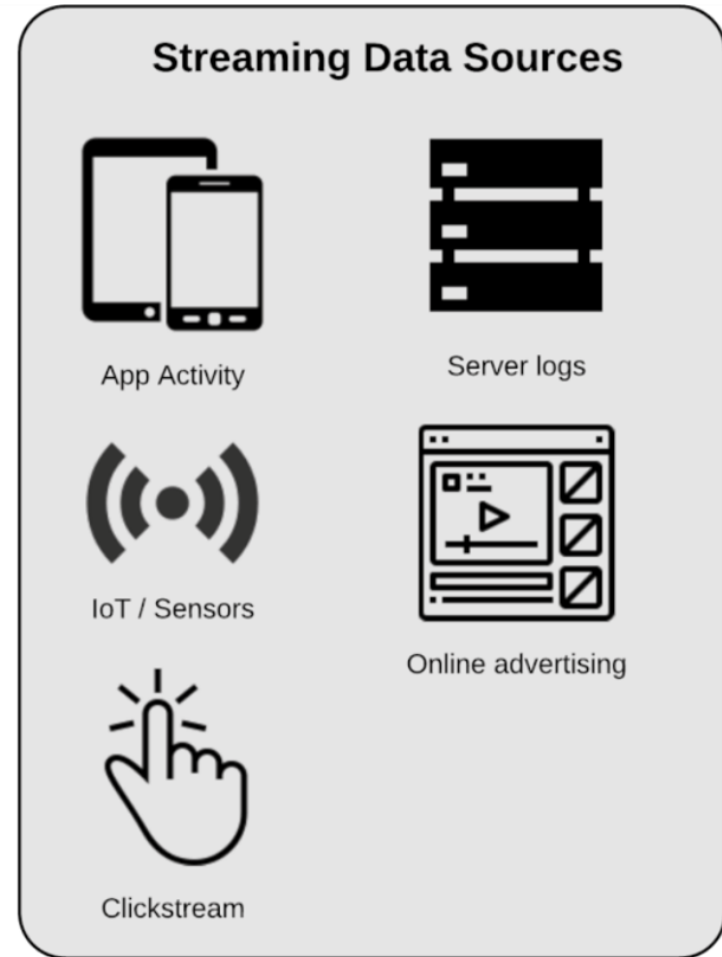
Distributed computing. Spark Streaming

Plan:

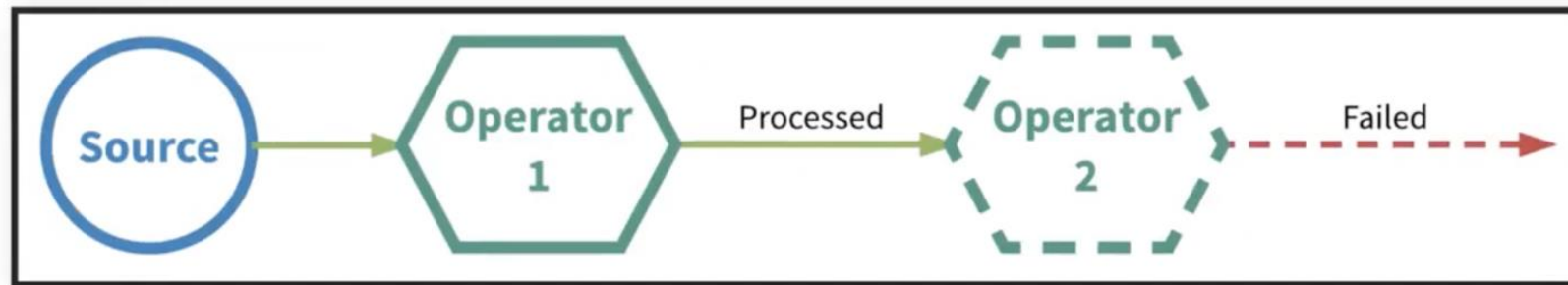
- Streaming data, where to find it
- Failure scenarios. Handling data loss scenarios
- Lambda architecture
- Spark Streaming & DStream. Structured Streaming
- Integration with MLlib

Streaming data and where to find it?

- IoT Sensors
- Server logs and security logs
- Real-time advertising
- Clickstream data from applications and websites

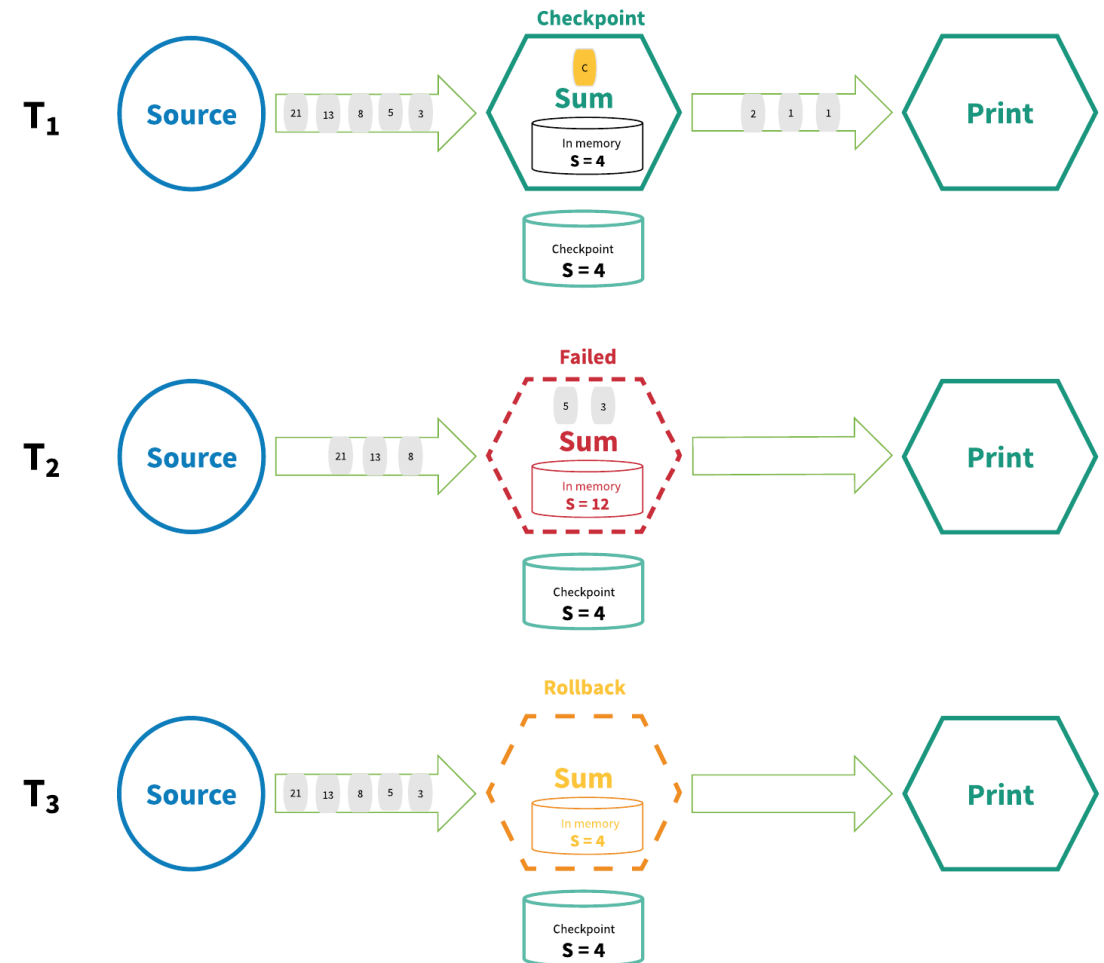


Failure scenarios. Failure during processing. At-most-once

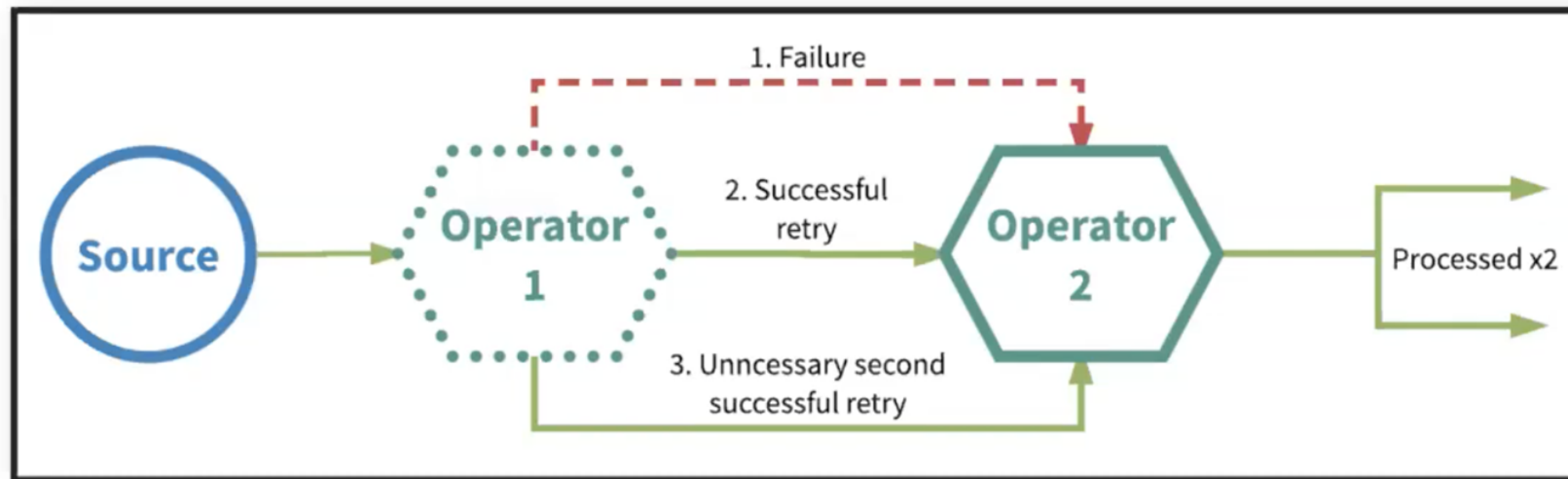


Failure scenarios. Distributed Snapshot

- The state for each operator in the streaming application is periodically checkpointed
- In the event of a failure, all the state is rolled back to the most recent globally consistent checkpoint
- **Sources are reset to the offset corresponding to the checkpoint**
- The application is rewound to its most recent consistent state, processing can then restart from that state



Failure scenarios. Failure during sending the processing result response. At-least-once



Failure scenarios. At-least-once event delivery. Message deduplication



Mathias Verraes

@mathiasverraes

 Follow



There are only two hard problems in distributed systems: 2. Exactly-once delivery 1. Guaranteed order of messages 2. Exactly-once delivery

RETWEETS

6,775

LIKES

4,727



10:40 AM - 14 Aug 2015



69



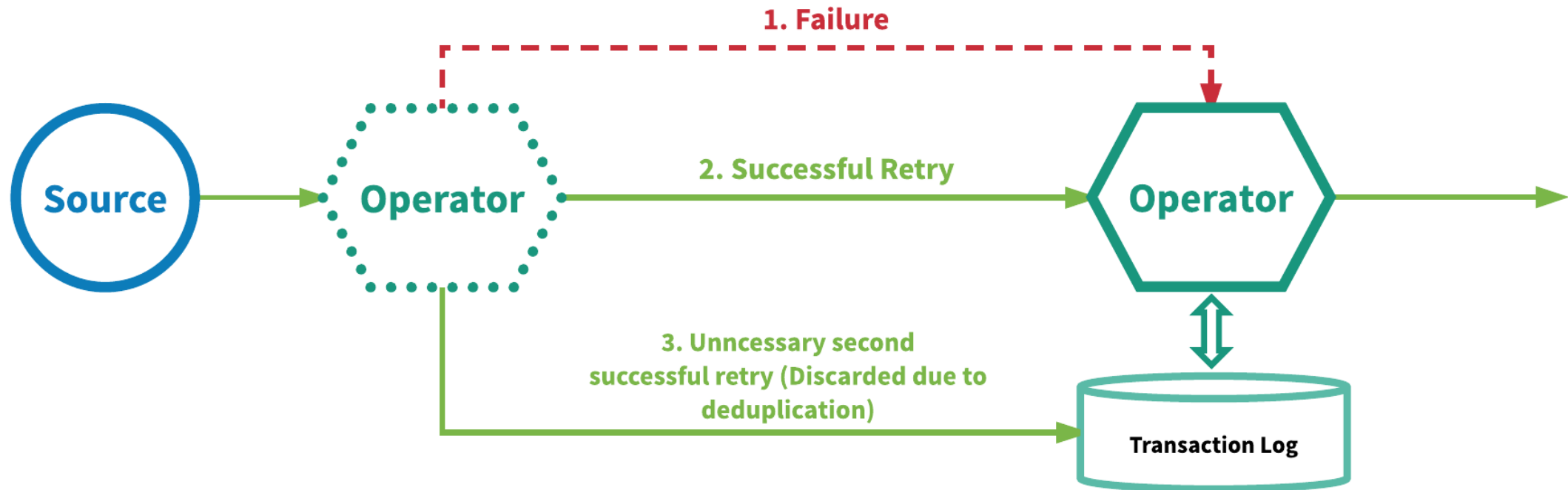
6.8K



4.7K



Failure scenarios. At-least-once event delivery. Message deduplication



End-to-end Exactly One Guarantee

Distributed snapshot:

- + *Little performance and resource overhead*
- *Larger impact to performance when recovering from failures*
- *Potential impact to performance increases as topology gets larger*

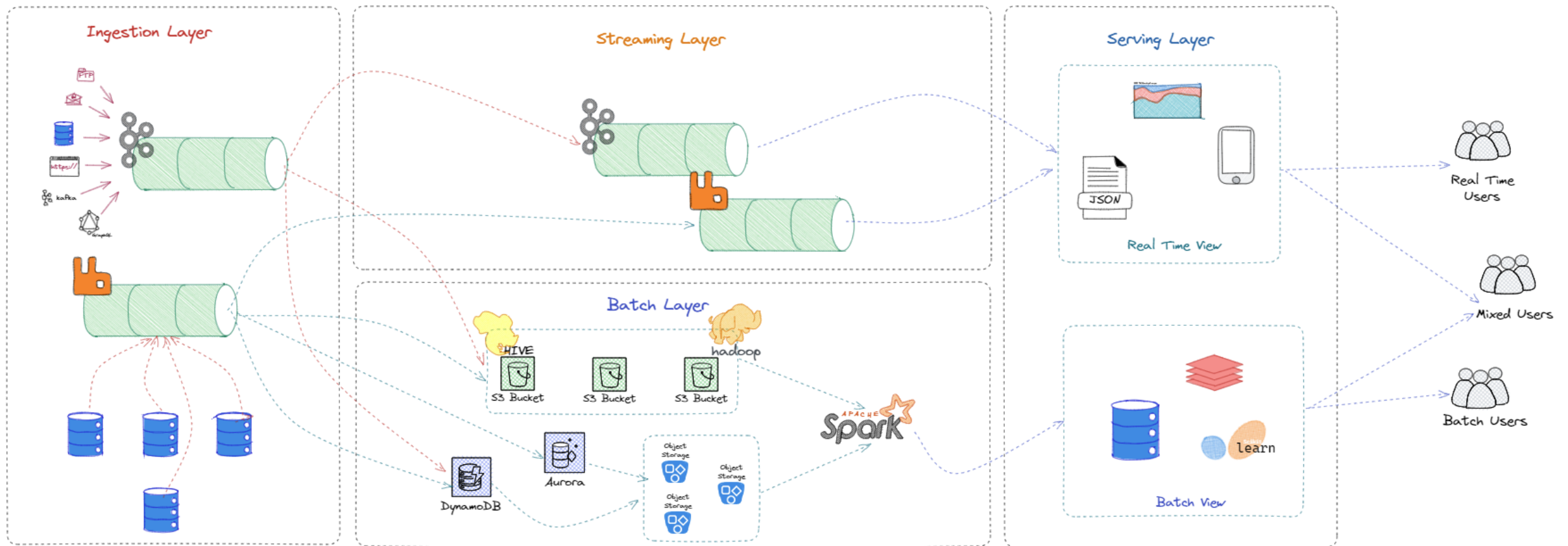
At-least-once delivery plus deduplication:

- + Performance impact of failures are localized
- + Impact of failures does not necessarily increase with the size of the topology
- Potentially need large amounts of storage and infrastructure to support
- Performance overhead for every event at every operator

In all cases, **the final state** of the data in the storage will be **exactly the same as if there were no failures** and each message was processed exactly once.

It does not guarantee exactly once processing!

Lambda architecture



(C) PradeepI.com

Lambda architecture combines **two separate data pipelines**:

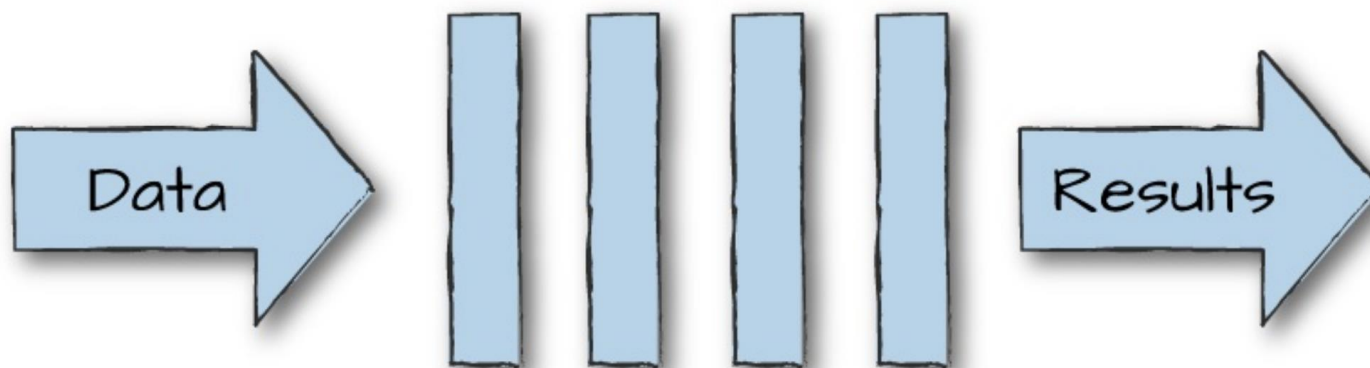
- **Batch Layer** for processing large volumes of historical data
- **Speed (streaming) Layer** for real-time stream processing for low-latency views of data

Spark Streaming & DStream

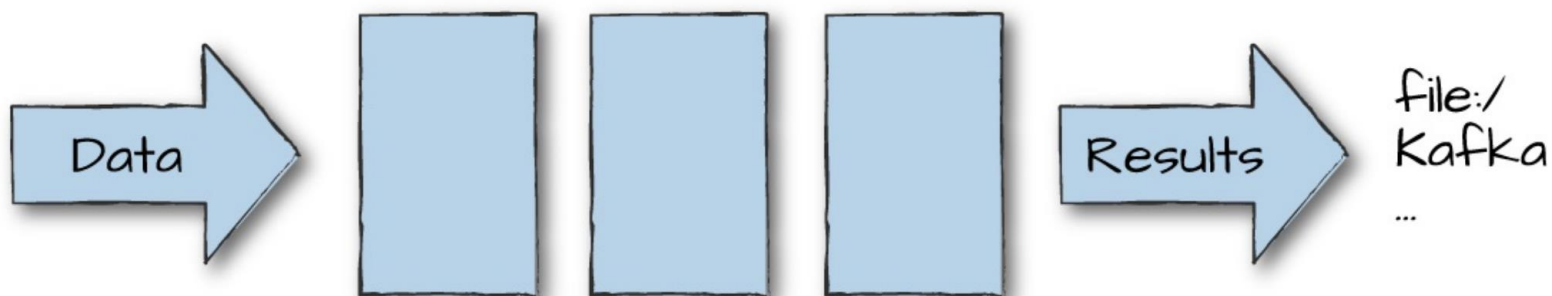
Spark Streaming



Spark Streaming. Micro Batches



One record at a time

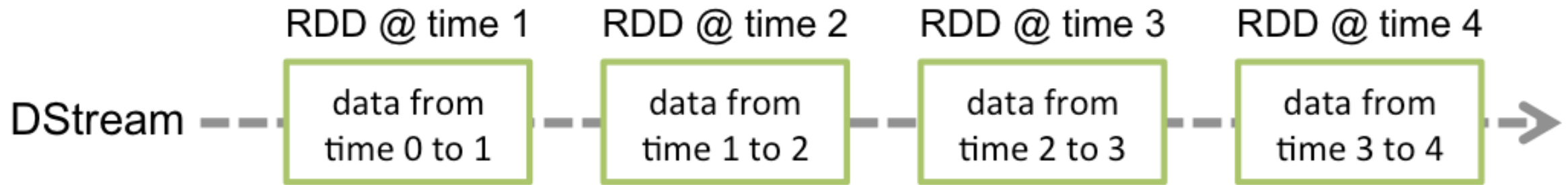


Microbatches of DataFrames

Spark Streaming. Programming model

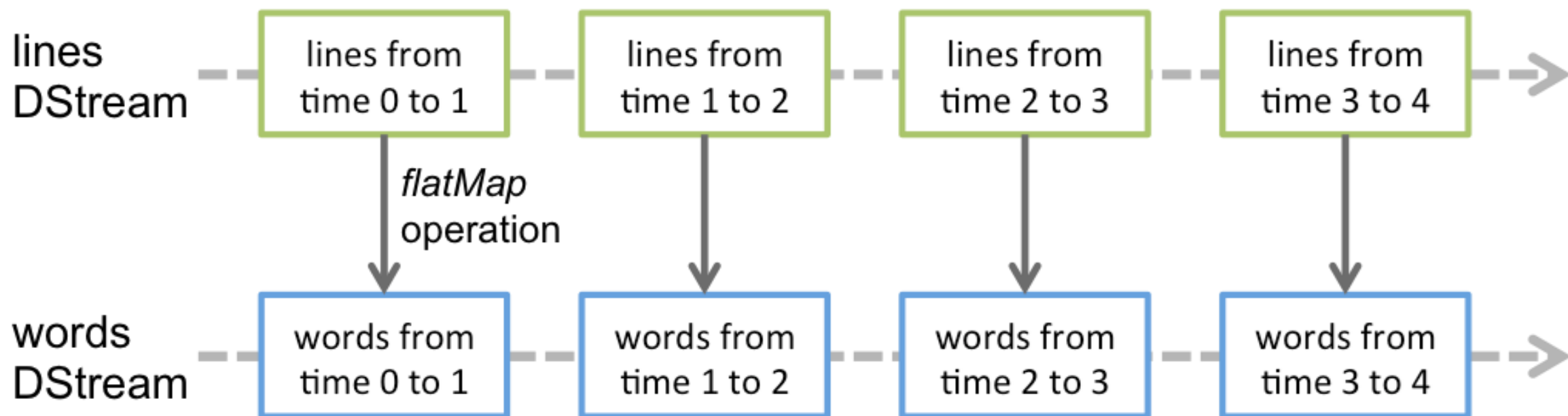


DStream. RDD Sequence

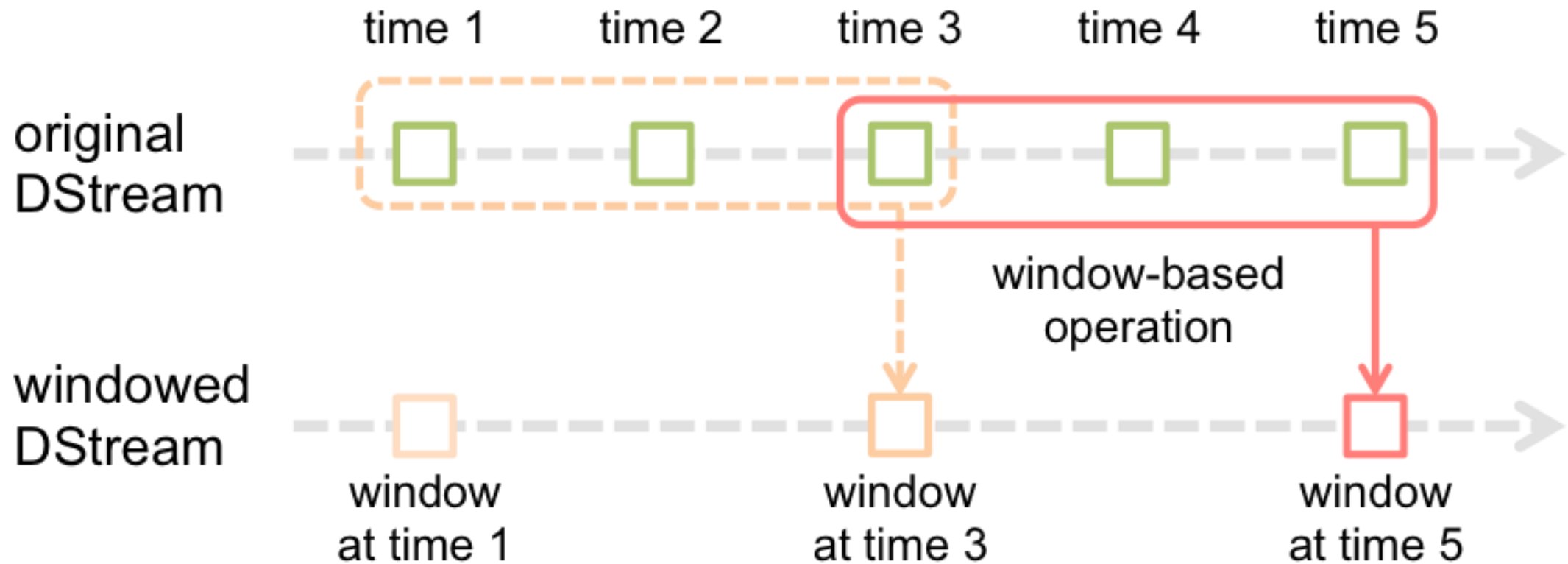


- **DStream (Discretized Stream)** — the basic abstraction of Spark Streaming
- Represents a continuous stream of data (from a source or after processing)
- Internally implemented as a sequence of RDDs
- Each RDD contains data for a fixed time interval

DStream. RDD Sequence

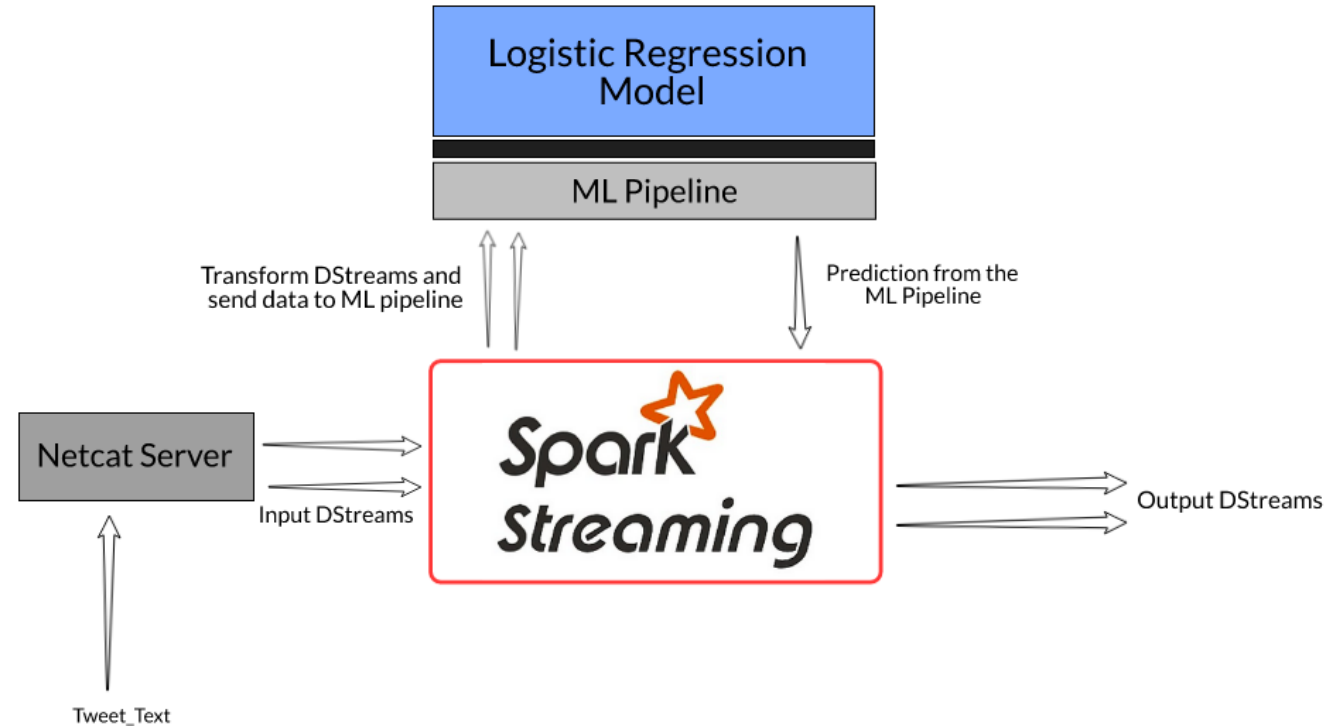


DStream. Window operations



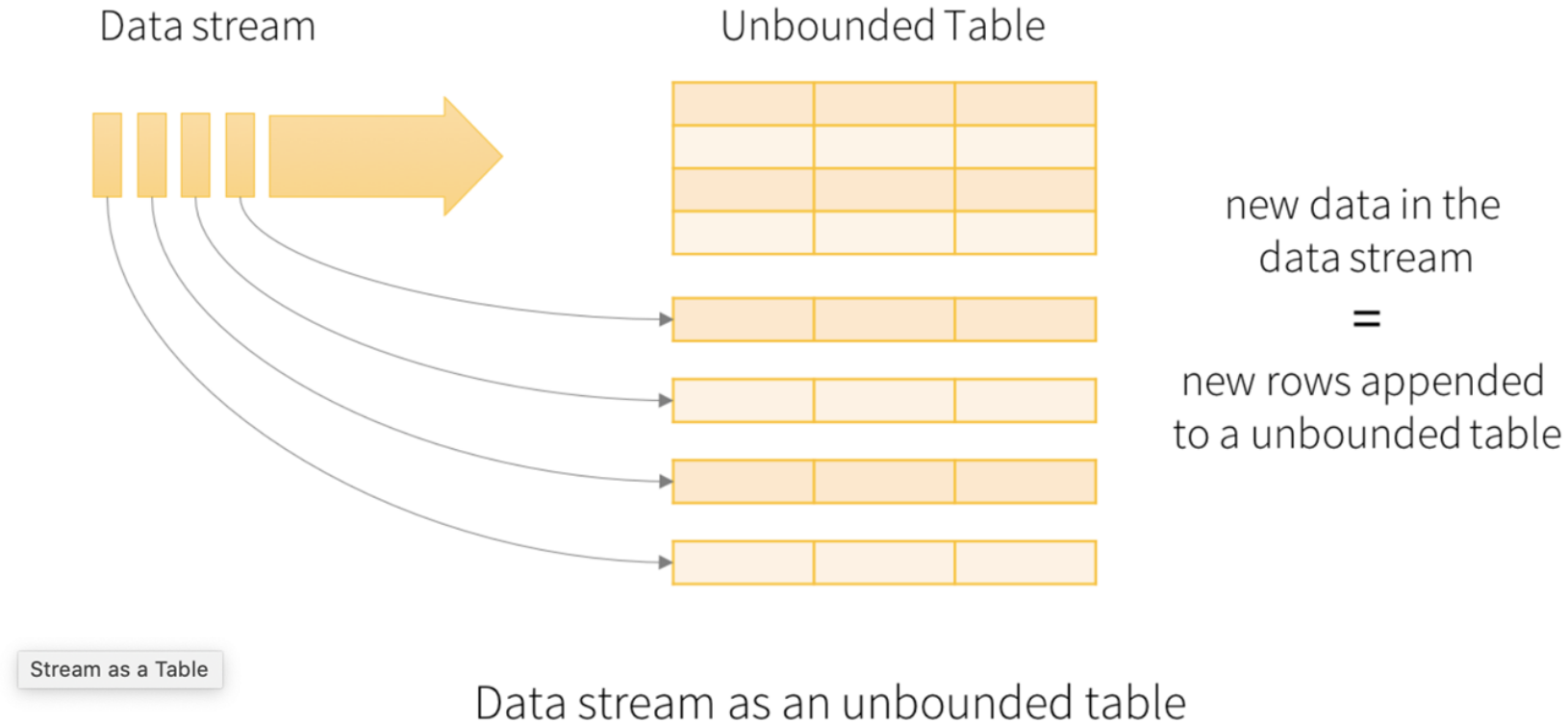
DStream. MLlib

- **Using MLlib:** easy integration of machine learning algorithms
- **Streaming learning:** simultaneous training and inference of the model (StreamingLinearRegression, KMeans)
- **Offline learning:** training on historical data and applying the model to the stream in real-time



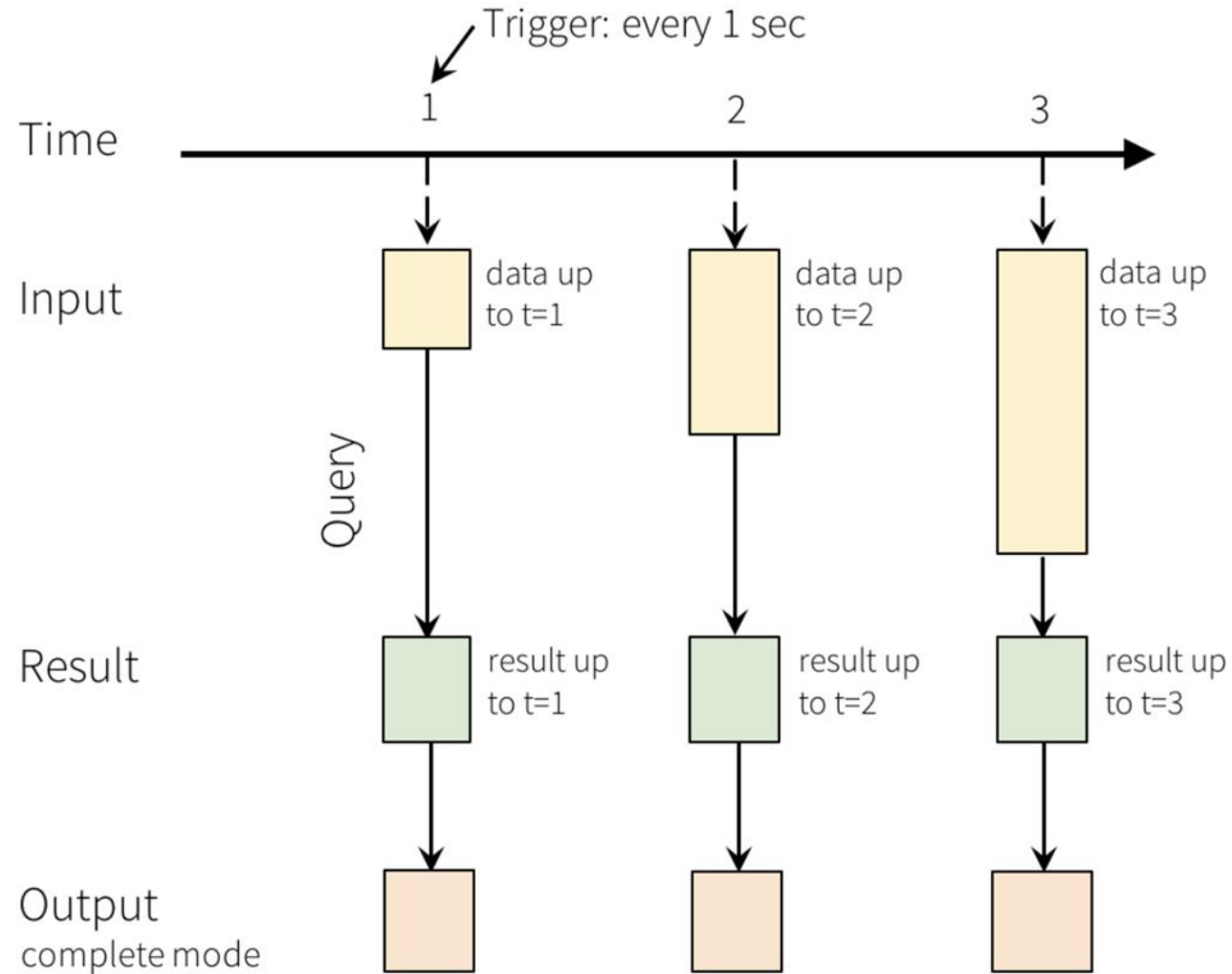
Structured Streaming & DataFrame

Structured Streaming. Programming model



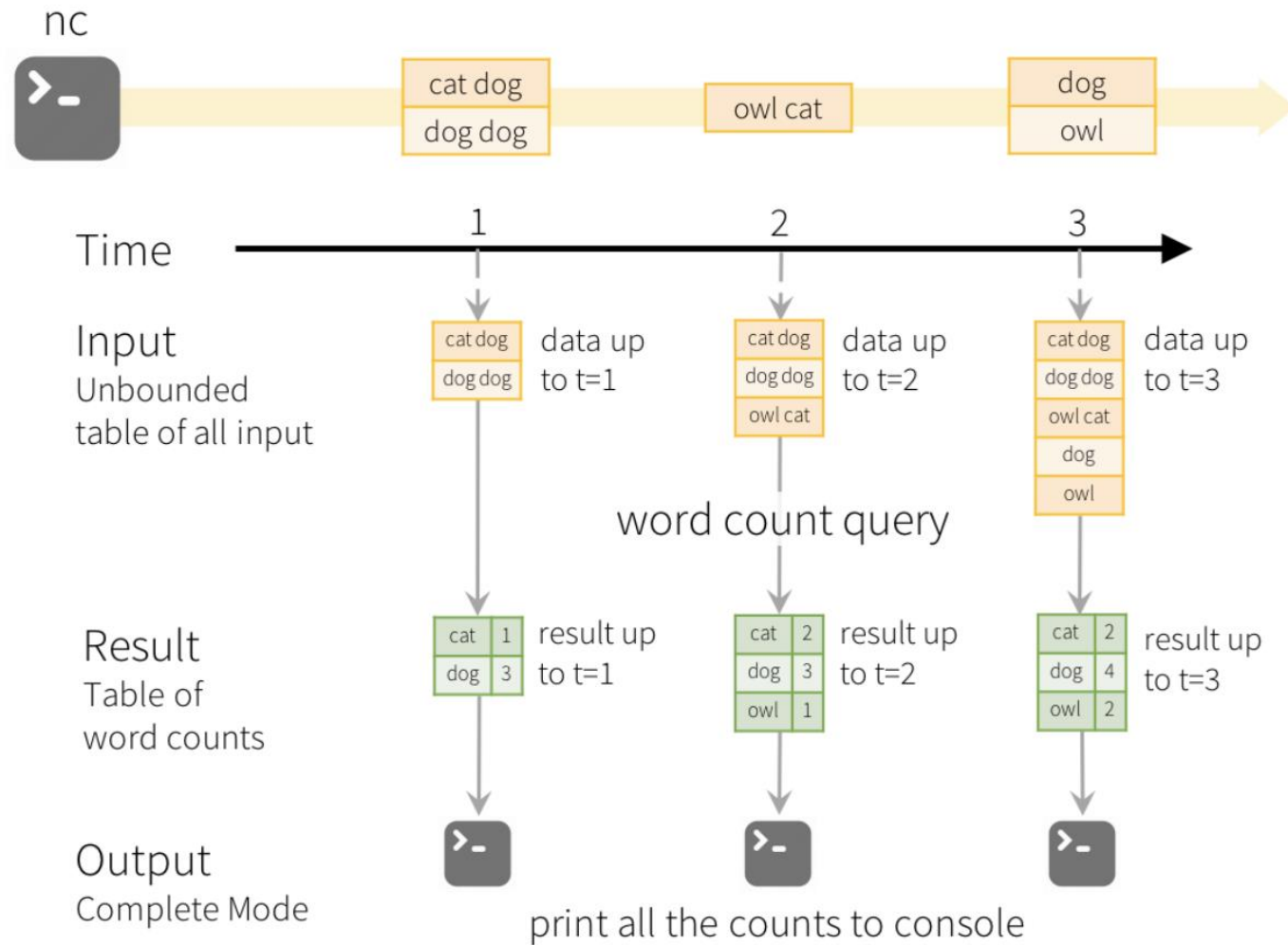
- **Key idea:** the data stream is treated as a constantly growing table
- **Unification:** the stream processing model is close to the batch processing model
- **API:** standard queries are used, just like for static data
- **Execution:** Spark runs them as incremental queries against the Unbounded Table

Structured Streaming. Programming model



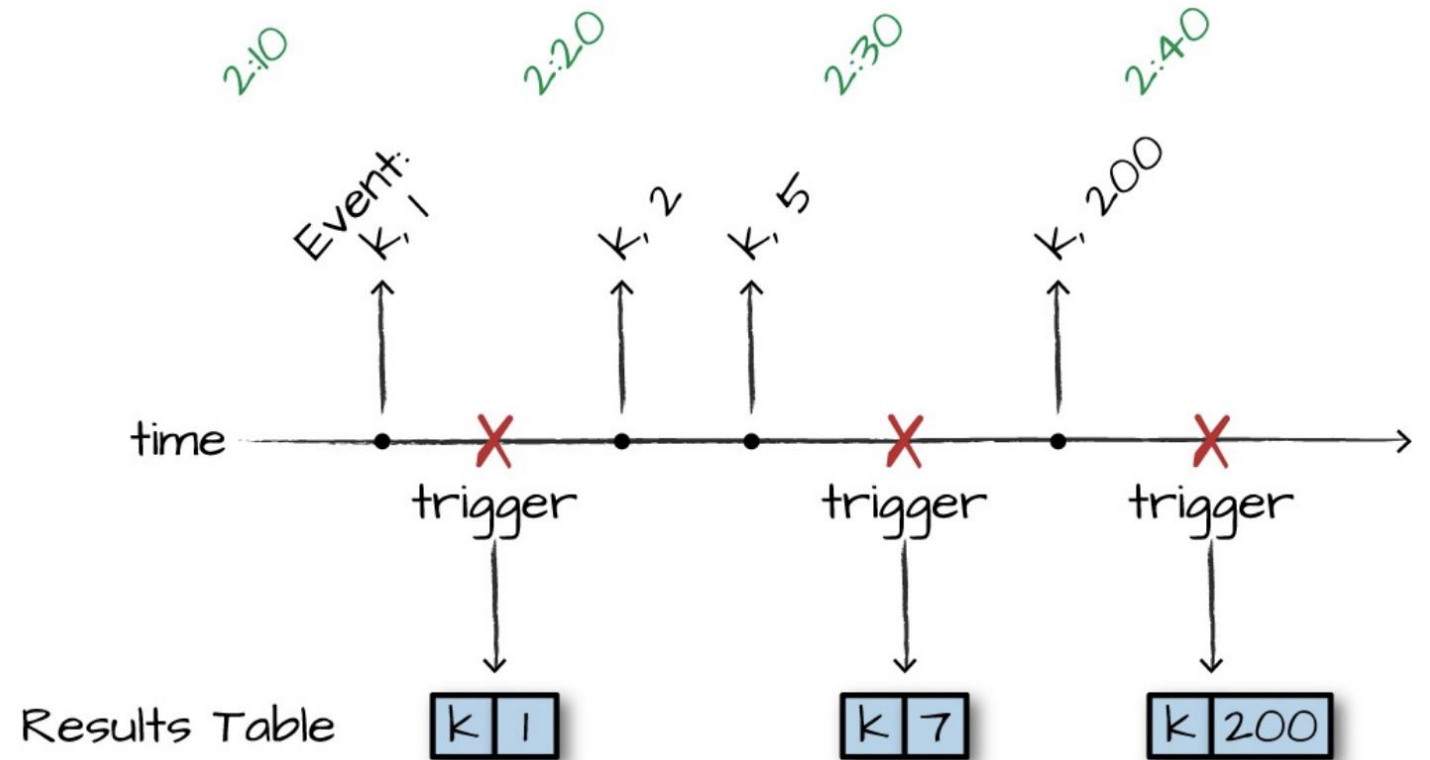
Programming Model for Structured Streaming

Structured Streaming. Example

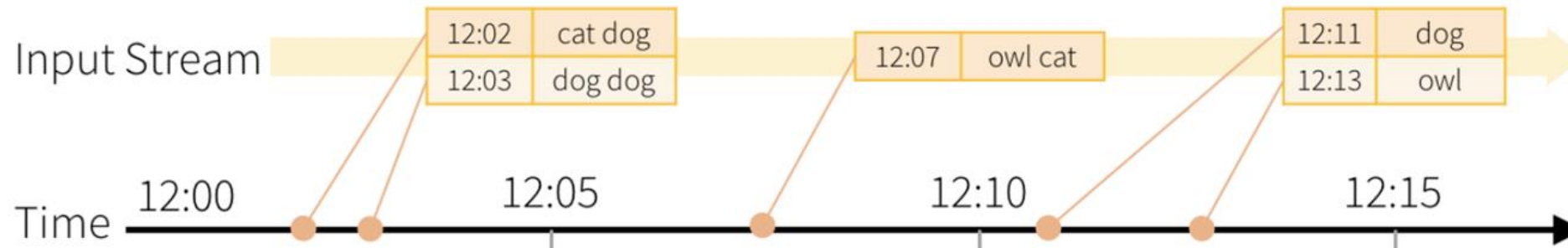


Model of the Quick Example

Structured Streaming. Event Processing



Structured Streaming. Event Processing



Result Tables
after 5 minute triggers

12:00 - 12:10	cat	1
12:00 - 12:10	dog	3

12:00 - 12:10	cat	2
12:00 - 12:10	dog	3
12:00 - 12:10	owl	1
12:05 - 12:15	cat	1
12:05 - 12:15	owl	1

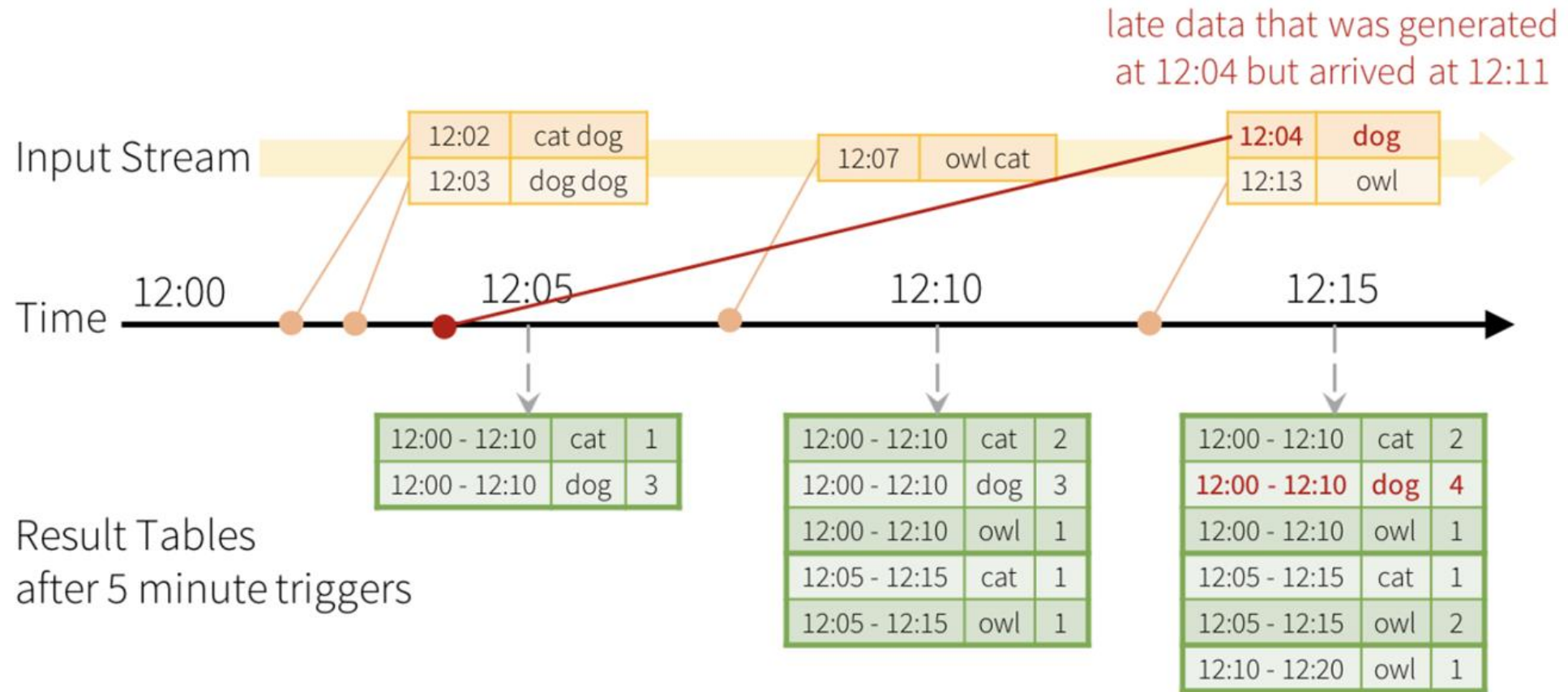
counts incremented for windows
12:00 - 12:10 and 12:05 - 12:15

12:00 - 12:10	cat	2
12:00 - 12:10	dog	3
12:00 - 12:10	owl	1
12:05 - 12:15	cat	1
12:05 - 12:15	owl	2
12:05 - 12:15	dog	1
12:10 - 12:20	dog	1
12:10 - 12:20	owl	1

counts incremented for windows
12:05 - 12:15 and 12:10 - 12:20

Windowed Grouped Aggregation
with 10 min windows, sliding every 5 mins

Structured Streaming. Late Data



counts incremented only for window 12:00 - 12:10

Late data handling in
Windowed Grouped Aggregation

Structured Streaming. Watermarking

